# STUDYSHARE

Project Documentation

| | |
|---|---|
| **Project Type** | Student study-material sharing web application |
| **Technology** | HTML, CSS, JavaScript, Bootstrap, Node.js, Express.js, MongoDB, Mongoose |
| **Prepared by** | _____ |
| **Course / Class** | _____ |
| **Submission Date** | _____ |

A web-based platform designed to help students upload, discover, and download useful academic notes.

# 1. Introduction

StudyShare is a web application that provides a shared space for students to exchange academic study material. Students can create accounts, sign in, upload notes with subject and course details, and browse approved resources. An administrator reviews uploaded materials before they become available in the public notes collection.

## 1.1 Purpose

The purpose of StudyShare is to make study resources easier to organize and access. Instead of relying only on private messages or scattered files, students can find materials by title, subject, semester, and branch, while the review process helps manage the material published on the platform.

## 1.2 Objectives

- Provide a central place to share academic notes and documents.
- Allow students to register and log in securely using password hashing.
- Collect useful note metadata such as title, subject, semester, branch, and description.
- Restrict public browsing to notes approved by an administrator.
- Provide note search and filtering, downloadable files, and download counts.
- Give administrators a page to review pending uploads and approve or reject them.

# 2. Project Scope

The current project is a browser-based web application served by an Express.js backend. The frontend pages are stored in the public directory and served by the same Node.js server. MongoDB stores user and note records, while uploaded document files are stored in the server's uploads directory.

## Main user roles

| Role | Main responsibilities |
| --- | --- |
| Guest / visitor | Open the home page, browse approved notes, and access registration or login pages. |
| Registered student | Log in, view the dashboard and profile, upload supported study files, and download approved notes. |
| Administrator | View all uploaded notes and approve or reject notes awaiting review. |

# 3. Technology Stack

| Technology | Use in the project |
| --- | --- |
| HTML5 | Defines page structure for home, login, registration, notes, upload, dashboard, profile, and admin pages. |
| CSS3 | Provides custom styling, layout, and visual presentation through public/css/style.css. |
| JavaScript | Handles form submission, API calls, note display, filtering, session-aware page behavior, and admin actions. |

| Technology | Use in the project |
|---|---|
| Bootstrap 5.3.3 | Provides responsive layout components and common interface styling. |
| Node.js | Runs the server-side JavaScript application. |
| Express.js | Provides routing, middleware, static file serving, and API endpoints. |
| MongoDB + Mongoose | Stores user accounts and note metadata using schema-based models. |
| bcryptjs | Hashes passwords during registration and compares password hashes during login. |
| express-session | Maintains login session information on the server. |
| Multer | Accepts document uploads, validates extensions, and saves files to the uploads directory. |
| dotenv / CORS | Loads environment configuration and enables Cross-Origin Resource Sharing middleware. |

# 4. Main Features

## 4.1 Account registration and login

- Registration collects the user's name, email, and password.
- The backend checks required fields and checks whether the email already exists.
- Passwords are hashed with bcryptjs before being saved in MongoDB.
- Login checks the email and compares the entered password with the stored hash.
- Successful login creates a session containing the user ID, name, email, and administrator flag.
- The profile endpoint returns the current user's data without returning the password field; logout destroys the session.

## 4.2 Browse and search notes

- The notes page requests notes from the backend and displays approved notes as cards.
- Each card shows note title, subject, semester, branch, uploader, description, and download count.
- Search matches the note title or subject.
- Semester and branch filters can be combined with the search field, and a reset control clears filters.
- Visitors can download an approved note through its download link.

## 4.3 Upload study material

- Uploading requires a logged-in user.
- Required metadata: title, subject, semester, and branch. Description is optional.
- Accepted extensions in the backend are PDF, DOC, DOCX, PPT, and PPTX.
- The configured maximum upload size is 10 MB per file.
- Uploaded files receive a generated server-side filename to reduce filename collisions.
- New note records are assigned the status 'pending' and are not shown in the public approved-notes list until approved.

### 4.4 Administrator review

- The administrator page lists all notes, uploader information, and current review status.
- Administrators can approve or reject notes whose status is pending.
- Approval changes the note status to 'approved'; rejection changes it to 'rejected'.
- Administrator routes are protected by session-based administrator middleware.

### 4.5 Dashboard and profile

The dashboard checks the current session and redirects unauthenticated users to the login page. It displays the signed-in user's name and adds the Admin Panel card when the user account has administrator privileges. The profile page displays account details and account type.

# 5. System Architecture

StudyShare follows a simple three-part architecture:

| Layer | Components | Responsibility |
|---|---|---|
| Presentation layer | HTML pages, CSS, Bootstrap, browser JavaScript | Displays forms and pages, gathers input, calls API routes, and renders results. |
| Application layer | Node.js, Express.js, route modules, middleware | Validates requests, checks session permissions, processes uploads, and coordinates database operations. |
| Data and file layer | MongoDB/Mongoose models and uploads/ folder | Stores user and note records in MongoDB and stores the uploaded document files on disk. |

### Typical upload flow

Student signs in → completes the upload form → browser sends multipart form data → authentication middleware checks the session → Multer validates and saves the file → backend saves note metadata with pending status in MongoDB → administrator reviews the note → approved notes appear in Browse Notes.

# 6. Database Design

### 6.1 User collection

| Field | Type / rule | Purpose |
|---|---|---|
| name | String, required | User's display name. |
| email | String, required, unique | Identifies the account and prevents duplicate email addresses. |
| password | String, required | Stores the password hash, not the original password. |
| isAdmin | Boolean, default false | Identifies administrator accounts. |
| timestamps | Created automatically | Mongoose adds createdAt and updatedAt. |

### 6.2 Note collection

| Field | Type / rule | Purpose |
|---|---|---|
| title, subject, semester, branch | String, required | Describes and classifies the study material. |
| description | String, optional | Additional information about the note. |
| fileName, filePath | String, required | Original display filename and stored file path. |
| uploadedBy | ObjectId reference to User, required | Links a note to its uploader. |
| downloadCount | Number, default 0 | Tracks the number of download requests. |
| status | String, default pending | Review state: pending, approved, or rejected. |
| timestamps | Created automatically | Records creation and last update time. |

# 7. Project Folder Structure

The main source files in the supplied project archive are organized as follows:

```
StudyShare/
■■■ server.js
■■■ package.json
■■■ .env
■■■ middleware/
■   ■■■ authMiddleware.js
■   ■■■ adminMiddleware.js
■■■ models/
■   ■■■ User.js
■   ■■■ note.js
■■■ routes/
■   ■■■ authRoutes.js
■   ■■■ noteRoutes.js
■   ■■■ adminRoutes.js
■■■ public/
■   ■■■ index.html
■   ■■■ register.html
■   ■■■ login.html
■   ■■■ dashboard.html
■   ■■■ notes.html
■   ■■■ upload.html
■   ■■■ profile.html
■   ■■■ admin.html
■   ■■■ 404.html
■   ■■■ css/style.css
■   ■■■ js/
■       ■■■ register.js
■       ■■■ login.js
■       ■■■ dashboard.js
■       ■■■ notes.js
■       ■■■ upload.js
■       ■■■ admin.js
■■■ uploads/
```

Note: The archive also contains node_modules and sample uploaded PDF files. node_modules can be regenerated from package.json and normally should not be included in a source-code submission ZIP.

# 8. API Endpoints

| Method | Endpoint | Purpose / access |
|---|---|---|
| POST | /api/auth/register | Register a user; expects name, email, password. |
| POST | /api/auth/login | Authenticate a user and create a session. |
| GET | /api/auth/me | Return the signed-in user's profile; login required. |
| POST | /api/auth/logout | Destroy the current session. |
| GET | /api/notes | Return approved notes for browsing. |
| POST | /api/notes/upload | Upload a note and metadata; login required; multipart field name: noteFile. |
| GET | /api/notes/download/:id | Download a note by its database ID and increment download count. |
| GET | /api/admin/notes | List all notes; administrator required. |
| PUT | /api/admin/notes/:id/approve | Approve a note; administrator required. |
| PUT | /api/admin/notes/:id/reject | Reject a note; administrator required. |

# 9. Installation and Execution

## Prerequisites

- Node.js and npm installed on the computer.
- A MongoDB database, local or hosted, with a connection string.
- The StudyShare project folder extracted from the ZIP archive.

## Setup steps

- Extract the project ZIP and open a terminal inside the StudyShare folder.
- Install dependencies by running: npm install
- Create or update the .env file with MONGODB_URI, PORT, and a secure session-secret variable. Do not publish real credentials.
- Confirm that the MongoDB connection string points to the intended database and that the database user has suitable permissions.
- Start in development mode with: npm run dev. Alternatively, use npm start.
- Open http://localhost:5000 in the browser, unless a different PORT is configured.
- Register a user and test login, note browsing, uploading, and logout. To test the admin panel, an account must have isAdmin set to true in the database by a trusted administrator.

## Environment configuration example (use your own values)

```
MONGODB_URI=mongodb_connection_string
PORT=5000
```

```
SESSION_SECRET=replace_with_a_long_random_secret
```

The supplied server code currently uses a literal session secret instead of reading SESSION_SECRET from the environment. Before deployment, update the session configuration to use process.env.SESSION_SECRET and configure secure cookie settings for HTTPS.

# 10. Testing Plan

| Test | Expected result |
|---|---|
| Register with all fields | Account is created and a success message is shown. |
| Register with missing fields | Request is rejected with a validation message. |
| Register an existing email | Request is rejected; duplicate account is not created. |
| Login with valid credentials | Session is created and dashboard opens. |
| Login with invalid credentials | Login is rejected with an error message. |
| Open dashboard without a session | User is redirected to login. |
| Upload without login | Backend rejects the request with an authentication error. |
| Upload an allowed document with required metadata | File is stored and note is saved as pending. |
| Upload an unsupported extension or a file over 10 MB | Upload is rejected by the upload validation/limit configuration. |
| Browse notes before approval | Pending note is not listed publicly. |
| Approve a pending note as admin | Status changes to approved and the note becomes publicly browseable. |
| Reject a pending note as admin | Status changes to rejected and the note remains unavailable in the public list. |
| Download an approved note | Browser downloads the file and download count increases. |
| Open admin routes as a non-admin | Request is rejected with an administrator-access error. |

# 11. Security and Maintenance Notes

- Keep .env files private. The supplied ZIP includes a .env file; inspect it before sharing and remove real database credentials or secrets from the archive. If credentials have already been exposed, rotate them.
- Use a strong random session secret from an environment variable. The current source sets the session secret to the literal text 'process.env.SESSION_SECRET', which does not read the environment variable.
- For production, configure HTTPS and secure session cookies, and review session-store persistence rather than relying on the default in-memory session store.
- Validate and sanitize user-provided text before displaying it. Frontend JavaScript currently inserts several note and uploader values into HTML template strings; use safe DOM text APIs or proper escaping to reduce cross-site scripting risk.
- Consider validating MIME type as well as extension, and scan uploaded files if the application is deployed for public use.

- Uploaded files are stored on the server's local disk. A production host may use an ephemeral filesystem; use persistent storage or object storage if uploaded files must survive redeployments.
- Add rate limiting and stronger password policy for public deployment, and ensure administrator privileges are granted only through a trusted process.

# 12. Known Limitations

- The current application stores files locally in an uploads directory, rather than in cloud/object storage.
- The implementation is intended as a student project and requires security hardening before public production use.
- The code's current session configuration uses a fixed literal secret and should be corrected as described above.
- The current notes API provides search and filtering in the browser after loading the approved notes list; large datasets may benefit from server-side search and pagination.
- Administrator account creation and privilege management are not exposed as a dedicated admin-management feature in the supplied routes.

# 13. Future Enhancements

- Add pagination and server-side search for large note collections.
- Allow students to see the status of their own uploads and manage their submissions.
- Add categories, tags, bookmarks, ratings, and report-content options.
- Store files in persistent cloud storage and add file previews where appropriate.
- Add password reset, email verification, and more robust account management.
- Add audit logs for administrator decisions and upload activity.
- Improve accessibility, form validation, automated tests, and deployment documentation.

# 14. Conclusion

StudyShare demonstrates a complete foundation for a student-focused study-material sharing platform. It combines a responsive web interface with an Express.js backend, MongoDB data storage, session-based authentication, document uploads, note discovery, download tracking, and administrator review. With the security and deployment improvements listed in this document, it can be extended into a more robust academic resource-sharing service.

# Appendix: Quick Reference

| Item | Value |
|---|---|
| Default local URL | http://localhost:5000 |
| Start command | npm start |
| Development command | npm run dev |
| Maximum configured upload size | 10 MB per file |
| Accepted file extensions | .pdf, .doc, .docx, .ppt, .pptx |

| Item | Value |
| --- | --- |
| Database models | User, Note |
| Main directories | middleware/, models/, routes/, public/, uploads/ |